Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025 (IB)

Lab. 10.1. Estructuras Enlazadas. Listas Enlazadas

Octubre 22 ,2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Una **lista enlazada** es una estructura de datos que permite almacenar una colección de datos de manera lineal.
Los **array** son estructuras que también permiten almacenar colección de datos, sin embargo, tienen limitaciones sustanciales, las operaciones *insertar* y *eliminar* se realizan a un alto costo, asimismo, tienen un tamaño fijo, que no permiten agregar más datos.
Las **listas de Python**, tienen tamaño flexible, pueden expandir o disminuir de tamaño, sin embargo, mantienen las limitaciones de los arrays con respecto a las operaciones *insertar* y *eliminar*. Con respecto al cambio de tamaño, tampoco es gratuito, se realiza creando un nuevo array cada vez que cambia de tamaño. Esto es debido a que las **listas de Python** están elaboradas en base a los array.

Asimismo, tanto los array y las listas de Python almacenan los datos de manera contigua, lo que acarrea problemas si el tamaño crece, podría ser difícil encontrar un área de memoria para asignarlo.

Las **listas enlazadas** mejoran las prestaciones de los **array** y las **listas de Python**, sin embargo, se pierde la posibilidad del acceso directo a los elementos, que sí lo tienen tanto los arrays como las listas de Python.

Hay muchas variedades de listas enlazadas, las más comunes son:
- Lista enlazada simple
- Lista doblemente enlazada
- Lista enlazada circular
- Lista doblemente enlazada circular

Implementación de Listas Enlazadas

La implementación de listas enlazadas está basada en el concepto de **nodo**. Un nodo es un objeto que contiene un **dato** y una **referencia (dirección)** a otro nodo.

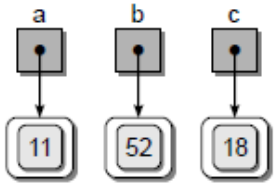
Supongamos que tenemos una clase básica que contiene un solo campo:

```
class Node :
      def __init__( self, element ) :
            self.data = element
```

Podemos crear varias instancias de esta clase, cada una almacenando un dato. En el siguiente ejemplo, creamos tres instancias, cada una almacenando un valor entero.

```
a = Node( 11 )
b = Node( 52 )
c = Node( 18 )
```

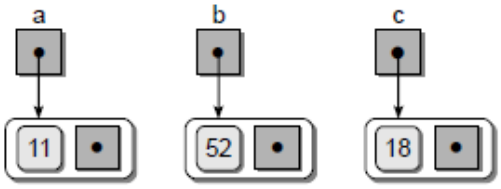
Como resultado habremos creado tres variables (referencias) y tres objetos, gráficamente tendremos:



Ahora, supongamos que agregamos un segundo campo (referencia) a la clase `Node`:

```
class Node :
      def __init__( self, element ) :
            self.data = element
            self.next = None
```

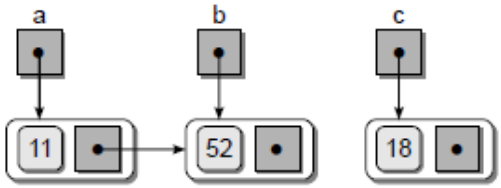
Los tres objetos del ejemplo anterior, ahora tendrán un segundo campo inicializado con referencia vacía.



Ya que el campo **`next`** puede contener una referencia a cualquier objeto **`Node`**, podemos asignarle una referencia que apunte a cualesquiera de los otros objetos **`Node`**. Por ejemplo, supongamos que asignamos **b** al campo **`next`** del objeto **a**:

```
a.next = b
```

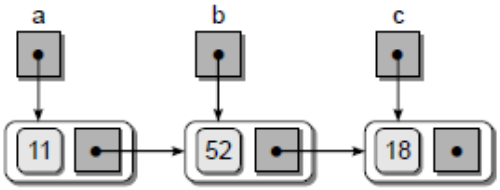
Como resultado obtenemos que el objeto **a** esté enlazado al objeto **b**:



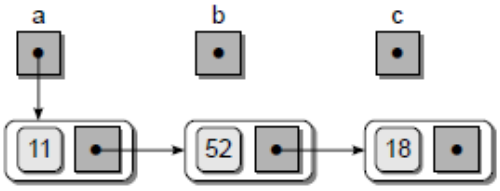
Y, finalmente enlazamos el objeto **b** a **c**:

```
b.next = c
```

Como resultado obtenemos una cadena de objetos, los objetos están enlazados, pero aun no es una lista enlazada:



Luego, podemos eliminar las referencias externas **b** y **c** asignándole `None` a cada uno, entonces obtenemos:

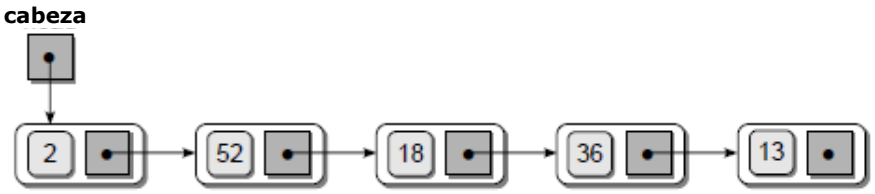


El resultado es una estructura llamada **lista enlazada**. Los objetos que antes estaban referenciados por **b** y **c**, aún son accesibles a través de **a**. Por ejemplo, podemos imprimir los datos de los tres objetos de la siguiente manera:

```
print( a.data )
print( a.next.data )
print( a.next.next.data )
```

Una **lista enlazada** es una estructura enlazada en el cual los nodos están conectados en secuencia para formar una lista lineal.

En el siguiente gráfico se muestra una lista enlazada con cinco nodos. El último nodo se llama nodo *cola* y tiene `None` como referencia. El primer nodo debe ser referenciado por una variable externa el cual provee el punto de entrada a la lista enlazada. Esta variable se llama apuntador o referencia al nodo cabeza.

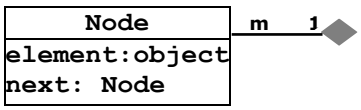


Recorrido de una Lista Enlazada

Recorrer una lista enlazada requiere la inicialización y ajuste de una variable de referencia externa temporal

```
def traversal( head ):
    curNode = head                  # curNode – variable temporal
    while curNode is not None :
          print(curNode.data)
          curNode = curNode.next
```

La clase `LinkedList`



LinkedList	
-head: Node -tail: Node -size: int	
__init__() #LinkedList()	Crea una lista enlazada vacía.
addFirst(e: object): None	Agrega un nuevo elemento a la cabeza de la lista
addLast(e: object): None	Agrega un nuevo elemento a la cola de la lista
getFirst(): object	Retorna el primer elemento de la lista.
getLast(): object	Retorna el último elemento de la lista.
removeFirst(): object	Elimina el primer elemento de la lista.
removeLast(): object	Elimina el último elemento de la lista.
add(e: object) : None	Lo mismo que `addLast(e)`.
insert(index:int,e:object):None	Agrega un nuevo elemento según el índice, index
clear(): None	Elimina todos los elementos de la lista.
contains(e: object): bool	Retorna True si la lista contiene al elemento
get(index: int) : object	Retorna el elemento según el índice especificado
indexOf(e: object) : int	Retorna el índice del primer elemento coincidente
isEmpty(): bool	Retorna True si la lista no contiene elementos
lastIndexOf(e: object) : int	Retorna el índice del último elemento coincidente
getSize(): int	Retorna la cantidad de elementos de la lista
remove(e: object): bool	Elimina el elemento de la lista
removeAt(ind ex: int) : object	Elimina el elemento especificado por el índice y retorna el elemento eliminado.
set(index:int,e:object):object	Coloca el elemento en el lugar especificado por el índice y retorna el elemento reemplazado
__iter__() : Iterator	Retorna un iterador para esta lista enlazada

1. Implemente la clase `Node`. (ya está implementado arriba). Implemente la clase `LinkedList` y los métodos `__init__()`, `addFirts()`, `getFirst()`,`getLast()` y `__str__()`. Luego adicionará los métodos que se vayan implementando en los siguientes ejercicios.

Implementando `addFirst(e)`. Adiciona un elemento al inicio de la lista.

```
def addFirst(self, e):
    newNode = Node(e)           # crea un nuevo nodo
    newNode.next = self.__head # enlaza el nuevo nodo con la cabeza
    self.__head = newNode       # head apunta al nuevo nodo
    self.__size += 1            # aumenta el tamaño de la lista

    if self.__tail == None:     # el nuevo nodo es el único nodo de la lista
        self.__tail = self.__head
```

2. Implemente el método **`addLast(self,e)`**. Considere dos casos, si la lista está vacía, tanto `head` como `tail` apuntarán al nuevo nodo, de lo contrario `tail` debería apuntar al nuevo nodo.

Implementando `insert(index, e)`. Adiciona un elemento en el lugar especificado por `index`.

```
def insert(self, index, e):
    if index == 0:
        self.addFirst(e)      # inserta al comienzo
    elif index >= self.__size:
        self.addLast(e)       # inserta al final
    else:                     # inserta en el medio
        current = self.__head
        for i in range(1, index):
            current = current.next
        temp = current.next
        current.next = Node(e)
        (current.next).next = temp
        self.__size += 1
```

3. Implemente el método **`removeFirst()`** que elimina el primer elemento de la lista. Considere el caso que la lista esté vacía. El tamaño debe disminuir en 1 si se elimina.

Implementando `removeLast()`. Remueve el último elemento de la lista enlazada.

```
def removeLast(self):
    if self.__size == 0:
        return None           # no hay elementos
    elif self.__size == 1:    # solo hay un elemento en la lista
        temp = self.__head
        self.__head = self.__tail = None  # la lista se vuelve vacía
        self.__size = 0
        return temp.element
    else:
        current = self.__head

        for i in range(self.__size - 2):
            current = current.next

        temp = self.__tail
        self.__tail = current
        self.__tail.next = None
        self.__size -= 1
        return temp.element
```

Ejercicios opcionales para resolver en casa.

4. Implemente el método **`removeAt(index)`**. El método encuentra el nodo en la ubicación especificada por index y lo elimina, devuelve el elemento eliminado. Considere cuatro casos.
   - Si **`index`** está fuera del rango de la lista, debe de devolver **`None`**
   - Si **`index`** es 0, debe de invocar a **`removeFirst()`** para eliminar el primer nodo
   - Si **`index`** es **`size-1`**, debe de invocar a **`removeLast()`** a fin de eliminar el último nodo
   - De lo contrario, ubicar al nodo en la posición especificada por **`index`**.

5. Implemente la clase **`LinkedListIterator`**. La implementación va a continuación. ¿por qué necesitamos un iterador específico?, ¿por qué no podemos usar el bucle `for` para recorrer directamente los elementos de la lista?. Pruebe el uso de `__iter__()`.

```
class LinkedListIterator:
   def __init__(self, head):
       self.current = head

   def __next__(self):
       if self.current == None:
           raise StopIteration
       else:
           element = self.current.element
           self.current = self.current.next
        return element
```

6. Implemente la clase **`LinkedList`** completa. La clase debe de contener al menos los siguientes métodos:
   a) __init()__()
   b) getFirst(self), devuelve el primer nodo de la lista (elemento cabeza)
   c) getLast(self), devuelve el último elemento de la lista
   d) addFirst(self,e), adiciona un elemento al inicio de la lista (ya está implementado arriba)
   e) addLast(self,e), adiciona un elemento al final de la lista (implementado en el ejer. N° 2)
   f) add(sefl,e), lo mismo que addLast()
   g) insert(self, index, e), inserta un nuevo elemento en el lugar especificado por index. (ya está implementado arriba)
   h) removeFirst(self), remueve el primer nodo. Devuelve el objeto contenido en el nodo eliminado. (implementado en el ejercicio N° 3)
   i) removeLast(self), remueve el último nodo. Devuelve el objeto contenido en el nodo eliminado. (implementado arriba)
   j) removeAt(self,index), remueve el elemento de la posición especificada por index. Devuelve el elemento que fue eliminado de la lista. (implementado en el ejercicio N° 4)
   k) isEmpty(self), devuelve True si la lista está vacía
   l) getSize(self), devuelve el tamaño de la lista
   m) __str__(sef), muestra los elementos de la lista encerrados entre corchetes y separados por comas (ejemplo: [Canada, America, Russia, France])
   n) clear(self), limpia la lista
   o) __iter__(self), retorna un iterador para la lista. El iterador debe de ser implementado en una clase separada. Si existe la clase **`LinkedListIterator`**, la implementación de __iter__ sería:

```
def __iter__(self):
      return LinkedListIterator(self.head)
```

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab10.1.**